SR UNIVERSITY

School of Computer Science and Artificial Intelligence

Lab Assignment # 13.2

Program	**: B. Tech (CSE)**
Specialization	**: -**
Course Title	**: AI Assisted Coding**
Course Code	**: 23CS002PC304**
Semester	**II**
Academic Session	**: 2025-2026**
Name of Student	**: P.Eshwar**
Enrollment No.	**: 2403A51L26**
Batch No.	**51**
Date	**: 10/03/26**

Submission Starts here

Screenshots:

Task Description #1(Refactoring – Eliminating Repetitive Logic)

Prompt:
Refactor the following Python program to eliminate repetitive logic.

Tasks:
1. Identify the repeated calculations in the code.
2. Encapsulate the repeated logic into reusable functions.
3. Ensure the program output remains the same after refactoring.
4. Add proper docstrings explaining the purpose, parameters, and return values of each function.

Original Code:
```
length = 8
width = 4
print("Area:", length * width)
print("Perimeter:", 2 * (length + width))

length = 12
width = 6
print("Area:", length * width)
print("Perimeter:", 2 * (length + width))
```

SR UNIVERSITY

The refactored solution should improve code readability, maintainability, and avoid duplication by using function

C: > Users > Eshwar > OneDrive > Desktop > python > ai_coding.py > ...

```python
# Refactor the following Python program to eliminate repetitive logic.

# Tasks:
# 1. Identify the repeated calculations in the code.
# 2. Encapsulate the repeated logic into reusable functions.
# 3. Ensure the program output remains the same after refactoring.
# 4. Add proper docstrings explaining the purpose, parameters, and return values of each function.

# Original Code:
# length = 8
# width = 4
# print("Area:", length * width)
# print("Perimeter:", 2 * (length + width))

# length = 12
# width = 6
# print("Area:", length * width)
# print("Perimeter:", 2 * (length + width))

# The refactored solution should improve code readability, maintainability, and avoid duplication by using functions.
def calculate_area(length, width):
    """
    Calculate the area of a rectangle.

    Parameters:
    length (float): The length of the rectangle.
    width (float): The width of the rectangle.

    Returns:
    float: The area of the rectangle.
    """
    return length * width

def calculate_perimeter(length, width):
    """
    Calculate the perimeter of a rectangle.

```

Copilot

C: > Users > Eshwar > OneDrive > Desktop > python > ai_coding.py > ...

```python

def calculate_perimeter(length, width):
    """
    Calculate the perimeter of a rectangle.

    Parameters:
    length (float): The length of the rectangle.
    width (float): The width of the rectangle.

    Returns:
    float: The perimeter of the rectangle.
    """
    return 2 * (length + width)
# Define the dimensions for the rectangles
rectangles = [(8, 4), (12, 6)]
# Calculate and print the area and perimeter for each rectangle
for length, width in rectangles:
    area = calculate_area(length, width)
    perimeter = calculate_perimeter(length, width)
    print(f"Area: {area}")
    print(f"Perimeter: {perimeter}")


```

Output:

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS

PS C:\Users\Eshwar\AppData\Local\Programs\Microsoft VS Code> & "C:\Program Files\Python314\python.exe" c:/Users/Eshwar/OneDrive/Desktop/python/ai_coding.py
Area: 32
Perimeter: 24
Area: 72
Perimeter: 36
```

Explaination:

- The refactored code defines two functions, `calculate_area` and `calculate_perimeter`, to encapsulate the logic for calculating the area and perimeter of a rectangle.
- The program iterates over a list of rectangle dimensions, calling the functions for each set of dimensions, thus eliminating repetitive code and improving readability.

Task Description #2 (Refactoring – Modularizing Inline Calculations)

Prompt:

Refactor the following Python script to modularize repeated inline calculations.

Tasks:

1. Identify the repeated calculation patterns in the code.

2. Extract the repeated logic into a reusable function.

3. Ensure the function has proper docstrings explaining its purpose, parameters, and return value.

4. Improve the readability and maintainability of the code.

5. Ensure the program output remains the same after refactoring.

Original Code:

```
amount = 1000

discount = amount * 0.10

final_amount = amount - discount

print("Final Amount:", final_amount)

amount = 2000

discount = amount * 0.10

final_amount = amount - discount
```

print("Final Amount:", final_amount)

The final solution should use a reusable function to calculate the final amount after applying the discount and print the result for different inputs

```
C: > Users > Eshwar > OneDrive > Desktop > python > ai_coding.py > ...
def calculate_final_amount(amount):
    """
    Calculate the final amount after applying a 10% discount.

    Parameters:
    amount (float): The original amount before discount.

    Returns:
    float: The final amount after discount.
    """
    discount = amount * 0.10
    final_amount = amount - discount
    return final_amount


# Example usage
amount1 = 1000
final_amount1 = calculate_final_amount(amount1)
print("Final Amount for $1000:", final_amount1)
amount2 = 2000
final_amount2 = calculate_final_amount(amount2)
print("Final Amount for $2000:", final_amount2)
```

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\Eshwar\AppData\Local\Programs\Microsoft VS Code> & "C:\Program Files\Python314\python.exe" c:/Users/Eshwar
Final Amount for $1000: 900.0
Final Amount for $2000: 1800.0
PS C:\Users\Eshwar\AppData\Local\Programs\Microsoft VS Code>
```

Explaination:

- The code contains repeated calculations for **discount and final amount**, which reduces readability and maintainability.
- Refactoring extracts this repeated logic into a **reusable function** that calculates the final amount after applying the discount.
- This makes the program **cleaner, modular, and easier to reuse** while keeping the output the same.

Task Description #3 (Refactoring – Improving Loop and Conditional Performance)

Prompt:

Refactor the following Python program to improve loop and conditional performance.

Tasks:

1. Analyze the code and identify inefficient nested loops and repeated conditional checks.

2. Suggest a more efficient data structure or logic (for example using sets or dictionaries for faster lookups).

3. Refactor the program to reduce time complexity while preserving the same behavior.

4. Add proper docstrings explaining the optimization.

5. Compare execution efficiency before and after refactoring using simple timing (for example using the time module).

Original Code:

```
students = ["Anil", "Bhavya", "Charan", "Divya"]

check_list = ["Charan", "Esha", "Bhavya"]


for name in check_list:
    exists = False
    for student in students:
        if name == student:
            exists = True
    if exists:
        print(name, "found")
    else:
        print(name, "not found")
```

The final program should:

- Use optimized lookup logic
- Maintain the same output
- Show a simple performance comparison between the original and optimized versions.

```python
import time
def original_code():
    students = ["Anil", "Bhavya", "Charan", "Divya"]
    check_list = ["Charan", "Esha", "Bhavya"]

    for name in check_list:
        exists = False
        for student in students:
            if name == student:
                exists = True
        if exists:
            print(name, "found")
        else:
            print(name, "not found")
def optimized_code():
    students = {"Anil", "Bhavya", "Charan", "Divya"}  # Using a set for O(1) lookups
    check_list = ["Charan", "Esha", "Bhavya"]

    for name in check_list:
        if name in students:  # Directly checking membership in the set
            print(name, "found")
        else:
            print(name, "not found")
# Timing the original code
start_time = time.time()
original_code()
end_time = time.time()
print(f"Original code execution time: {end_time - start_time:.6f} seconds")
# Timing the optimized code
start_time = time.time()
optimized_code()
end_time = time.time()
print(f"Optimized code execution time: {end_time - start_time:.6f} seconds")
```

Output:

```
PS C:\Users\Eshwar\AppData\Local\Programs\Microsoft VS Code> & "C:\Program Files\Python314\python.exe" c:/Users/Eshwar/OneDrive/
Charan found
Esha not found
Bhavya found
Original code execution time: 0.000746 seconds
Charan found
Esha not found
Bhavya found
Optimized code execution time: 0.000357 seconds
PS C:\Users\Eshwar\AppData\Local\Programs\Microsoft VS Code>
```

Explaination:

- This refactored code improves the performance of the original program by replacing the list of students with a set. Sets in Python provide O(1) average time complexity for membership checks, which significantly reduces the time taken to determine if a name exists in the list of students.

- In the original code, there is a nested loop where for each name in the check_list, it iterates through the students list to check for a match. This results in O(n*m) time complexity, where n is the length of check_list and m is the length of students.

- In the optimized code, we convert the students list into a set, allowing us to check for membership in O(1) time. This reduces the overall time complexity to O(n), where n is the length of check_list, as we only need to iterate through the check_list once.

Task Description-4 (Refactoring – Replacing Hardcoded Values

with Constants)

prompt:

Refactor the following Python program to replace hardcoded numeric values with named constants.

Tasks:

1. Identify hardcoded numeric values used directly in expressions.

2. Define meaningful constants (for example RATE and TIME) at the beginning of the program.

3. Modify the calculations to use these constants instead of hardcoded numbers.

4. Add proper comments or docstrings explaining the constants and calculations.

5. Ensure the program output remains the same after refactoring.

Original Code:

print("Simple Interest:", (1000 * 2 * 5) / 100)

print("Simple Interest:", (2000 * 2 * 5) / 100)

The final solution should improve readability and maintainability by using descriptive constants such as RATE and TIME while keeping the same output.

```
C: > Users > Eshwar > OneDrive > Desktop > python > ai_coding.py > ...
# Refactor the following Python program to replace hardcoded numeric values with named constants.

# Tasks:
# 1. Identify hardcoded numeric values used directly in expressions.
# 2. Define meaningful constants (for example RATE and TIME) at the beginning of the program.
# 3. Modify the calculations to use these constants instead of hardcoded numbers.
# 4. Add proper comments or docstrings explaining the constants and calculations.
# 5. Ensure the program output remains the same after refactoring.

# Original Code:
# print("Simple Interest:", (1000 * 2 * 5) / 100)
# print("Simple Interest:", (2000 * 2 * 5) / 100)

# The final solution should improve readability and maintainability by using descriptive constants such as RATE and TIME while keeping the same outp
# Refactored Code with Named Constants
# Define constants for the interest rate and time period
RATE = 2  # Interest rate in percentage
TIME = 5  # Time period in years
# Define constants for the principal amounts
PRINCIPAL_1 = 1000  # First principal amount
PRINCIPAL_2 = 2000  # Second principal amount
# Calculate and print simple interest for the first principal amount
simple_interest_1 = (PRINCIPAL_1 * RATE * TIME) / 100
print("Simple Interest for Principal 1:", simple_interest_1)
# Calculate and print simple interest for the second principal amount
simple_interest_2 = (PRINCIPAL_2 * RATE * TIME) / 100
print("Simple Interest for Principal 2:", simple_interest_2)
```

Output:

```
PS C:\Users\Eshwar\AppData\Local\Programs\Microsoft VS Code> & "C:\Program Files\Python314\python.exe" c:/Users/Eshwar/OneDrive/Desktop/python/ai_coding.py
Simple Interest for Principal 1: 100.0
Simple Interest for Principal 2: 200.0
PS C:\Users\Eshwar\AppData\Local\Programs\Microsoft VS Code>
```

Explaination:

- The program contains hardcoded numbers (2, 5, 100) directly in the formula, which reduces readability and flexibility.

- Refactoring replaces these numbers with named constants like RATE, TIME, and PERCENT defined at the beginning of the code.
- This makes the program easier to understand, maintain, and update while keeping the same output.

Task Description -5 (Re factoring – Enhancing Variable Naming

and Code Clarity)

Refactor the following Python program to improve variable naming and code clarity.

Prompt:

Tasks:

1. Identify non-descriptive variable names in the code.
2. Replace them with meaningful and descriptive variable names.
3. Add inline comments to explain the logic where necessary.
4. Ensure the functionality and program output remain the same.
5. Improve overall readability and maintainability of the code.

Original Code:

```
x = 50
y = 30
z = x + y
print(z)
```

The final solution should use clear variable names and include helpful comments while producing the same output.

```python
# Refactor the following Python program to improve variable naming and code clarity.

# Tasks:
# 1. Identify non-descriptive variable names in the code.
# 2. Replace them with meaningful and descriptive variable names.
# 3. Add inline comments to explain the logic where necessary.
# 4. Ensure the functionality and program output remain the same.
# 5. Improve overall readability and maintainability of the code.

# Original Code:
# x = 50
# y = 30
# z = x + y
# print(z)

# The final solution should use clear variable names and include helpful comments while producing the same output.
# Define the first number to be added
first_number = 50
# Define the second number to be added
second_number = 30
# Calculate the sum of the two numbers
total_sum = first_number + second_number
# Print the result to the console
print(total_sum)
```

Output:

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS

PS C:\Users\Eshwar\AppData\Local\Programs\Microsoft VS Code> & "C:\Program Files\Python314\python.exe" c:/Users/Eshwar/OneDrive/Desktop/python/ai_coding.py
80
PS C:\Users\Eshwar\AppData\Local\Programs\Microsoft VS Code>
```

Explaination:

- The original code uses unclear variable names (x, y, z) which makes the program harder to understand.
- Refactoring replaces them with meaningful variable names and adds comments to explain the logic.
- This improves code readability and maintainability while keeping the output the same.